

Unit 2 — Search, Games & CSP

Review deck for the test on Thu Oct 8

English for the exam · "ES:" tips in Spanish

Wiki: Problem Formulation · Uninformed / Greedy / A · Heuristics · Local Search ·
Minimax · Alpha-Beta · MCTS · Expectiminimax · CSP*

Problem formulation (5 parts)

1. **Initial state** — e.g., `In(Arad)`
2. **Actions(s)** — applicable moves
3. **Transition model** `Result(s, a)`
4. **Goal test** — e.g., `In(Bucharest)`
5. **Action cost** `c(s, a, s')`

Together they define the **state space** (a graph). Search keeps a **frontier** (generated, not yet expanded) and a **reached/explored** set to avoid loops.

ES: el nodo $\neq$ el estado. Un nodo guarda estado + padre + acción + costo g.

"The frontier order defines the algorithm"

Frontier ordered by...	Algorithm
FIFO queue	BFS
LIFO stack	DFS
$g(n)$ — path cost	UCS / Dijkstra
$h(n)$ — heuristic	Greedy best-first
$g(n) + h(n)$	A*

A* = UCS + greedy: set $h = 0 \rightarrow$ UCS; ignore $g \rightarrow$ greedy.

Goal test: BFS can test when **generating**; UCS and A* must test when **expanding** (popping).

Properties table (memorize)

	Complete	Optimal	Time	Space
BFS	yes (finite b)	if all costs equal	$O(b^d)$	$O(b^d)$
DFS	no (yes in finite spaces with cycle check)	no	$O(b^m)$	$O(b \cdot m)$
Depth-limited	no if $l < d$	no	$O(b^l)$	$O(b \cdot l)$
IDS	yes	if all costs equal	$O(b^d)$	$O(b \cdot d)$
UCS	yes (costs $\geq \epsilon > 0$)	yes	$O(b^{(1+\lceil C^*/\epsilon \rceil)})$	same
Greedy	no (yes finite + cycle check)	no	$O(b^m)$ worst	$O(b^m)$
A*	yes	yes (admissible / consistent h)	depends on h	$O(b^d)$

b = branching factor · d = depth of shallowest solution · m = max depth · C^* = optimal cost · ϵ = min action cost

ES: IDS genera solo ~11 % más nodos que BFS ($b = 10$, $d = 5$: 123 450 vs 111 110) con memoria lineal.

Heuristics

- $h(n)$ estimates the cost from n to the goal; $h(\text{goal}) = 0$.
- **Admissible:** never overestimates — $h(n) \leq h^*(n)$.
- **Consistent:** $h(n) \leq c(n, a, n') + h(n')$ (triangle inequality). Consistent $\Rightarrow$ admissible.
- Examples: straight-line distance (Romania); **misplaced tiles** h_1 and **Manhattan** h_2 (8-puzzle).
- $h_2 \geq h_1$ for every state $\rightarrow h_2$ **dominates** $h_1 \rightarrow A^*$ with h_2 expands no more nodes.

ES: admisible basta para A^* óptimo si el algoritmo **reabre** estados al encontrar un camino más barato; sin reabrir hace falta consistente (errata #4).

A* on Romania: Arad → Bucharest

Step	Expand	$f = g + h$	Frontier after (f)
1	Arad	$0 + 366 = 366$	Sibiu 393 · Timisoara 447 · Zerind 449
2	Sibiu	$140 + 253 = 393$	Rimnicu 413 · Fagaras 415 · ... · Oradea 671
3	Rimnicu Vilcea	$220 + 193 = 413$	Fagaras 415 · Pitesti 417 · ... · Craiova 526
4	Fagaras	$239 + 176 = 415$	Pitesti 417 · ... · Bucharest 450
5	Pitesti	$317 + 100 = 417$	Bucharest 418 (improved) · ...
6	pop Bucharest	$418 + 0$	goal → cost 418

Greedy (by h only) goes Sibiu → Fagaras → Bucharest = **450** (not optimal).

ES: Bucharest ya estaba en la frontera con 450, pero A* no se detiene hasta **sacarlo** de la frontera.

Real numbers from the homework

Problem	Algorithm	Result	Effort
Romania (10-city subgraph)	Dijkstra / A*	418 / 418	9 / 5 expanded
8-puzzle	BFS	6 moves	135 generated
8-puzzle	DFS (limit 10)	10 moves	484 generated
8-puzzle	Greedy, Manhattan	6 moves	15 generated
Sudoku	naive backtracking	solved	4 208 assignments
Sudoku	MRV + forward checking	solved	51 assignments

Local search & hill climbing

- Keep only the **current state**; move to a neighbor; no path memory → tiny memory, huge spaces.
- **Hill climbing**: move to the best neighbor; stop when none is better → stuck on **local maxima, ridges, plateaus**.
- Random 8-queens: plain hill climbing solves only **14 %**; with sideways moves **94 %**.
- Fixes: sideways moves, stochastic / first-choice, **random restarts**, local beam search, **simulated annealing**.

ES: búsqueda local = solo importa el estado final, no el camino (N-reinas, optimización).

Minimax

- Two-player, zero-sum, deterministic, perfect information.
- `MINIMAX(s)` = utility if terminal; **max** over children on MAX's turn; **min** on MIN's turn.
- Complete (finite tree), optimal against an optimal opponent; time $O(b^m)$, space $O(b \cdot m)$.
- Real games: **depth cutoff + evaluation function**.

AIMA Fig. 6.2:

```

      MAX
     /  |  \
  MIN 3 MIN 2 MIN 2
 3 12 8 2 4 6 14 5 2

```

→ root = 3 (move a1)

Alpha-beta pruning

- α = best value MAX can guarantee so far (lower bound) · β = best for MIN (upper bound).
- Stop exploring a node's children as soon as $\alpha \geq \beta$.
- **Same answer as minimax**, less work. Perfect ordering: $O(b^{(m/2)}) \rightarrow$ search **twice as deep**.

Tree	Root	Pruned leaves
AIMA [[3,12,8],[2,4,6],[14,5,2]]	3	4, 6
Practice 5 [[4,8,9],[3,7,1],[6,2,5]]	4	7, 1, 5
Practice 6 [[[3,5],[6,9]],[[1,2],[0,-1]]]	5	9 and subtree [0, -1]

ES: practica estos tres en el **Algorithm Lab** (pestaña *Games*).

MCTS and UCB1

Four steps, repeated: **selection** → **expansion** → **simulation (playout)** → **back-propagation**.

$UCB1(n) = U(n)/N(n) + C \cdot \sqrt{\ln N(\text{parent}) / N(n)}$ — exploitation + exploration

Practice 7: $N = 15$; children A 7/10, B 3/4, C 0/1.

- $C = 1 \rightarrow$ picks **C** (1.646): barely tried $\rightarrow$ explore.
- $C = 0.3 \rightarrow$ picks **B** (0.997): best win rate $\rightarrow$ exploit.

Used in Go ($b \approx 361$, no good evaluation function): AlphaGo / AlphaZero = MCTS + neural networks.

Expectiminimax (games of chance)

- Add **chance nodes**: value = probability-weighted **average** of children.
- Cost $O(b^m \cdot n^m)$ for n chance outcomes.
- Evaluation must be a *positive linear transformation* of win probability (order-preserving is not enough).

Practice 8: $a_1 \rightarrow 0.5 \cdot 2 + 0.5 \cdot 4 = \mathbf{3}$; $a_2 \rightarrow 0.5 \cdot 1 + 0.5 \cdot 8 = \mathbf{4.5} \rightarrow$ choose **a_2** .

Treating chance as MIN would wrongly choose a_1 .

CSP — constraint satisfaction

- **Variables** X , **domains** D , **constraints** C ; solution = complete + consistent assignment.
- **Backtracking** + heuristics:
 - **MRV** (fewest legal values) · **degree** (tie-break: most constraints)
 - **LCV** — try the value that rules out fewest options for neighbors
- **Forward checking**: after assigning, remove inconsistent values from neighbors.
- **AC-3 / MAC**: make every arc consistent; detects more failures earlier.
- Tree-structured CSP: $O(n \cdot d^2)$.

ES: MRV = "falla primero" (elige variable); LCV = "deja opciones" (elige valor).

CSP practice 9 (Australia, 3 colors)

WA = red, then Q = green $\rightarrow$ forward checking gives
NT = {blue}, SA = {blue}, NSW = {red, blue}, V = {r, g, b}.

- **MRV** picks NT or SA (1 value left).
- Forward checking does **not** notice NT and SA (neighbors) both only have blue; **AC-3 / MAC would** and would backtrack immediately.

Arc consistency, $X < Y$ with $X, Y \in \{1, 2, 3\} \rightarrow X = \{1, 2\}, Y = \{2, 3\}$.

Self-check (answer aloud, in English)

1. Why is UCS optimal but greedy is not?
2. Admissible vs. consistent — define both; which implies which?
3. Trace A* from Arad: when is Bucharest first generated, and when is it returned?
4. Run alpha-beta on $[[4,8,9],[3,7,1],[6,2,5]]$ — what is pruned?
5. Why does Go use MCTS instead of alpha-beta?
6. MRV vs. LCV — what does each choose?

Answers: wiki [study/practice-unit-2-search.md](#) · Algorithm Lab · Exam Drill quiz